

Chapter 2

apply object-oriented concepts (inheritance, polymorphism, design patterns, etc.) in

Data structures serve as the structural backbone of software systems, defining how memory is organized and manipulated. However, raw data structures written without proper abstraction mechanisms can become tightly coupled to specific programs, making code difficult to maintain, extend, or reuse. Object-oriented programming provides architectural principles that decouple abstract behavioral contracts from concrete physical layouts.

This chapter examines how key object-oriented concepts, specifically inheritance, polymorphism, and design patterns, are applied to data structure design. By leveraging these concepts, developers can construct modular data structure libraries that decouple client interfaces from underlying operational mechanics.

Understanding these abstraction techniques is essential for modern software development. Building upon the built-in data structures explored in Chapter 1, this material establishes the structural patterns used to build custom, extensible data structures in Chapter 3 and beyond.

2.1 Object-Oriented Abstraction and Abstract Data Types

In software development, an **abstract data type** (ADT) is a mathematical specification of a collection of data and the operations that can be performed on that data. An ADT defines *what* operations are available, their parameters, and their expected outcomes, without specifying *how* those operations are implemented in memory. A concrete data structure, by contrast, is the physical implementation of an ADT in code, specifying data fields, memory allocation strategies, and algorithmic routines.

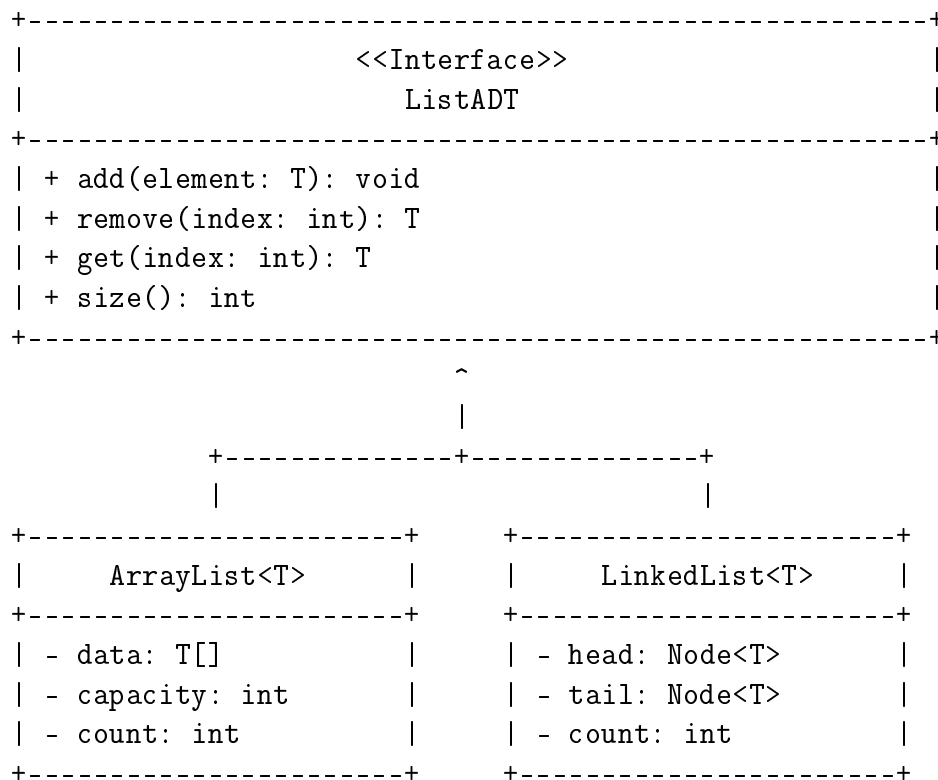
Object-oriented programming enforces ADT specifications through encapsulation and access control. Encapsulation isolates the internal memory representation of a data structure behind a public interface. Private fields prevent external code from directly modifying pointer references or array indices, preserving internal structural invariants.

2.1.1 Separating Specification from Implementation

Consider a sequence of elements accessible by index. The ADT specification defines operations such as insertion, removal, search, and size retrieval. Multiple distinct concrete data structures can satisfy this same specification:

- A **dynamic array**, which stores elements in contiguous memory and offers fast positional access.
- A linked list, which stores elements in disjoint nodes connected by pointers, offering fast insertions at node boundaries.

By defining an abstract base class or interface, client programs can operate directly on the abstraction. This allows software systems to substitute concrete representations without altering the client code that depends on them.



2.2 Inheritance and Polymorphism in Data Structure Design

Inheritance is an object-oriented mechanism where a child class acquires fields and methods from a parent class. In data structure engineering, inheritance serves two primary roles: code reuse and subtype specification.

Subtyping inheritance defines formal class hierarchies where child classes fulfill the contract of a parent type. Implementation inheritance allows derived classes to inherit common operational logic, reducing duplicate code across related structures.

Polymorphism enables objects of different concrete classes to respond to the same method call in class-specific ways. Through dynamic method dispatch, the runtime environment inspects the concrete type of an object and invokes the corresponding method implementation.

2.2.1 Class Hierarchies and Code Sharing

Inheritance hierarchies prevent redundancy by placing shared operational logic high in the class tree. An abstract base class can implement operations that depend only on other abstract primitives.

For example, an abstract list base class can provide a concrete implementation of an `isEmpty()` method that checks if `size() == 0`. Subclasses inheriting this base class receive `isEmpty()` automatically, needing only to implement the storage-specific `size()` method.

Table 2.1: Comparison of Inheritance and Composition in Data Structure Design

Dimension	Inheritance (“is-a”)	Composition (“has-a”)
Relationship	Establishes a rigid subtype hierarchy between parent and child classes.	Embeds an instance of another data structure as an internal field.
Coupling	High coupling; modifications to base classes affect all derived classes.	Low coupling; the encapsulated instance can be swapped independently.
Flexibility	Static; behavior is determined at compile time by class definitions.	Dynamic; contained objects and strategy delegates can be updated at runtime.
Primary Purpose	Defining shared interface contracts and inheriting shared implementations.	Building complex or restricted structures from simple components.

While inheritance creates tight coupling between base and child classes, composition embeds a data structure inside another class, delegating work to the internal object. Designing data structures often requires choosing between inheritance and composition to maintain flexibility.

2.3 Applying Design Patterns to Data Structures

Design patterns are reusable, generalized software engineering solutions to recurring design problems within a given domain. Applying design patterns to data structures yields modular, flexible implementations that adapt easily to changing requirements.

Four essential design patterns used extensively in data structure design are:

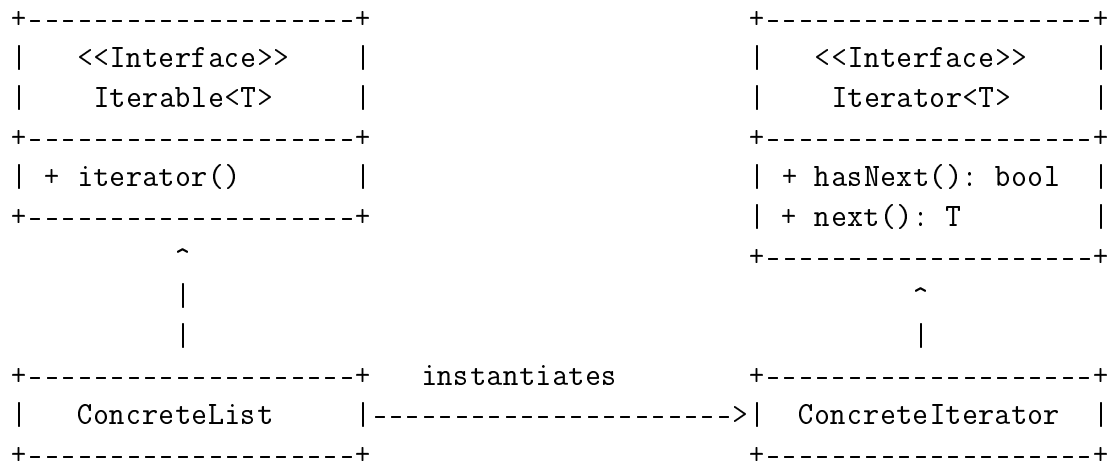
1. **Iterator Pattern:** Traverses aggregate elements sequentially without exposing internal structural representations.

2. **Factory Pattern:** Encapsulates object creation mechanics for nodes and structural components.
3. **Strategy Pattern:** Parameterizes key algorithmic behaviors within a data structure.
4. **Adapter Pattern:** Converts an existing data structure interface into an interface expected by client code.

2.3.1 The Iterator Pattern

The **iterator pattern** separates element traversal from the internal layout of a collection. Without an iterator, client code traversing a linked list must manually navigate pointers, while traversing an array requires array index management.

An iterator provides a unified, sequential interface featuring operations such as `hasNext()` and `next()`. Client code can iterate over arrays, trees, or graphs using identical loop structures.



The iterator preserves data structure integrity by preventing client code from modifying internal pointers directly during traversal.

2.3.2 The Factory Pattern

The **factory pattern** delegates object instantiation to dedicated methods or builder classes. In complex data structures, creating nodes or structural elements directly inside algorithms couples the structure to concrete node classes.

For example, a balanced search tree class can use a factory method, `createNode(key, value)`, to generate tree nodes. Subclasses override this factory method to instantiate specialized nodes, such as Red-Black nodes containing color bits, without altering the core search tree logic.

2.3.3 The Strategy Pattern

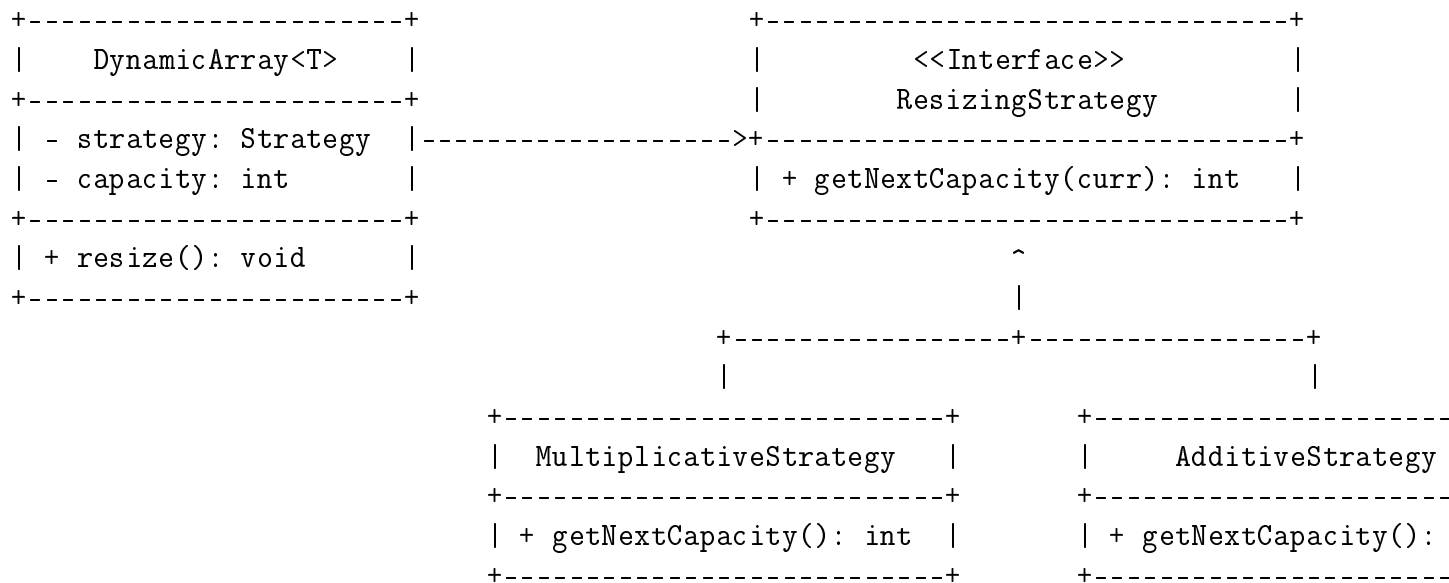
The **strategy pattern** encapsulates interchangeable algorithms into separate classes conforming to a shared interface. Data structures often rely on external algorithms to drive

internal behavior, including:

- Custom comparison operations for sorting and heap ordering.
- Alternative hash functions for hash tables.
- Dynamic resizing strategies for dynamic arrays.

By passing a strategy object to a data structure during construction, the structure's runtime behavior can be changed without modifying its source code.

For instance, a dynamic array can accept a growth strategy interface. One strategy can grow capacity multiplicatively by doubling ($1.5\times$ or $2.0\times$), while another increases capacity linearly by fixed increments.



2.3.4 The Adapter Pattern

The **adapter pattern** converts the interface of an existing class into a different interface expected by a client. This pattern is commonly used to implement restricted access data structures, such as Stacks and Queues, using existing linear structures.

Consider implementing a Stack ADT using a dynamic array. A Stack requires `push()`, `pop()`, and `peek()` operations, whereas a dynamic array exposes indexed access operations like `addAt()`, `removeAt()`, and `get()`. The Stack adapter class wraps an internal dynamic array instance, translating stack operations into array actions.

```

class StackAdapter<T>:
    private list: DynamicArray<T>

    constructor():
        list = new DynamicArray<T>()

    method push(item: T): void:

```

```

list.addLast(item)

method pop(): T:
    if list.isEmpty():
        throw UnderflowException
    return list.removeLast()

method peek(): T:
    if list.isEmpty():
        throw UnderflowException
    return list.get(list.size() - 1)

```

2.4 Performance Trade-Offs and Architectural Considerations

Applying object-oriented abstractions to data structures introduces performance trade-offs that must be evaluated against engineering advantages.

Table 2.2: Summary of Core Design Patterns in Data Structure Design

Pattern	Category	Structural Context	Primary Benefit
Iterator	Behavioral	Collection element traversal.	Decouples collection traversal from internal layout.
Factory	Creational	Node and container creation.	Encapsulates object creation and enables specialization.
Strategy	Behavioral	Hashing, growth, and sorting routines.	Enables runtime selection of dynamic algorithms.
Adapter	Structural	Stack, Queue, and Deque construction.	Reuses existing container code via interface transformation.

2.4.1 Virtual Method Overhead and Memory Layout

Dynamic polymorphism requires runtime method dispatch. In languages that compile to virtual machine bytecode or native binaries, polymorphic calls use virtual method tables (vtables) to locate method pointers at runtime. This indirect call mechanism introduces minor latency compared to direct function calls and can prevent CPU compiler optimizations such as inline expansion.

Additionally, object-oriented abstractions introduce memory overhead:

- **Object headers:** Language runtimes attach metadata, such as type indicators and synchronization locks, to every object instance.

- Indirection and cache performance: Storing collections of objects via pointer references can cause CPU cache misses due to disjoint memory locations, compared to contiguous arrays of primitive data types.

Contiguous Primitive Array Layout (Cache-Friendly):

```
+-----+-----+-----+-----+
| Data 1 | Data 2 | Data 3 | Data 4 |
+-----+-----+-----+-----+
```

Reference-Based Object Layout (Pointer Indirection):

```
+-----+-----+-----+-----+
| Ref 1  | Ref 2  | Ref 3  | Ref 4  |
+-----+-----+-----+-----+
|         |         |         |         |
|   v     |   v     |   v     |   v     |
+-----+ +-----+ +-----+ +-----+
|Obj 1|  |Obj 2|  |Obj 3|  |Obj 4|
+-----+ +-----+ +-----+ +-----+
```

Engineers must weigh clean architectural design against low-level execution speed. For general application development, object-oriented abstractions improve code maintainability, safety, and reusability. In performance-critical systems, direct memory layouts and simplified procedural abstractions may be preferred.

2.5 Summary

Object-oriented programming concepts provide tools for engineering reusable data structure libraries. Abstract data types separate interface definitions from concrete storage mechanics. Inheritance establishes formal type hierarchies and shares common implementations, while polymorphism allows applications to manipulate collections generically. Design patterns like Iterator, Factory, Strategy, and Adapter solve common architectural problems, improving data structure flexibility and reuse.

These principles pave the way for Chapter 3, where we will build fundamental data structures, including stacks, queues, linked lists, trees, and graphs, from the ground up using object-oriented principles.

Tutorial Questions

1. Explain the distinction between an Abstract Data Type (ADT) and a concrete data structure. Provide two distinct concrete data structure examples for a single ADT specification.
2. Describe how encapsulation helps protect the internal structural invariants of a complex data structure. What vulnerabilities arise when internal representation details are exposed?

3. Compare inheritance (“is-a”) and composition (“has-a”) as techniques for constructing new data structures. In what scenario is composition preferred over inheritance?
4. Illustrate the Iterator Pattern using a class diagram or pseudocode. How does the iterator pattern decouple client code from the physical storage representation of a collection?
5. An engineer needs to implement a bounded Queue structure using an existing `DynamicArray` class. Identify which design pattern should be applied, write pseudocode for the class wrapper, and explain how the wrapper enforces queue behavior.
6. Discuss the Strategy Pattern in the context of dynamic array resizing. Design a scenario where switching dynamic array growth strategies at runtime improves performance or memory efficiency.
7. Analyze the performance costs associated with applying polymorphism and abstract interface abstractions to high-performance data structures. Identify two computational or memory overheads introduced by object-oriented abstraction layers.